

Universidad de Antioquia
Facultad de Ingeniería — Informática II

Marvel Boxing

Videojuego de boxeo arcade en un universo de superhéroes

Desarrollado en C++ con interfaz gráfica Qt

Integrantes: Nicolás Anacona
Andrés Maca

Profesor: Anibal guerra

Curso: Informática II

Fecha: 05 de junio de 2026

0 Índice

1. Introducción	3
2. Planteamiento del problema	3
3. Definición general y objetivos	3
3.1. Idea general del videojuego	3
3.2. Objetivo general	4
3.3. Objetivos específicos	4
4. Especificación de requerimientos	4
4.1. Requisitos funcionales	4
4.2. Requisitos no funcionales	5
4.3. Restricciones y dependencias	5
5. Metodología y planificación	5
6. Diseño y arquitectura del sistema	6
6.1. Separación por capas	6
6.2. Descripción de clases principales	7
7. Desarrollo e implementación	7
7.1. Programación orientada a objetos	7
7.2. Memoria dinámica	8
7.3. Contenedores seleccionados y justificación	8
7.4. Interfaz gráfica y conexiones en Qt	9
8. Modelos físicos implementados	9
8.1. Gravedad lunar con flotación	9
8.2. Movimiento oscilatorio de plataformas	9
8.3. Caída vertical de objetos con altura simulada	10
8.4. Colisiones y rango de impacto	10
9. Agente inteligente	10
9.1. Percepción	10
9.2. Razonamiento	10
9.3. Acción	10
9.4. Aprendizaje	11
10.Dificultad	11
11.Cambios frente a los momentos anteriores	12
12.Procedimientos de prueba	12
12.1. Plan de pruebas	12
12.2. Resultados de prueba	13
12.3. Controles	13
13.Resultados y discusión	13

14. Conclusiones	14
Referencias	14
A. Estructura de carpetas	15
B. Fragmento representativo de herencia propia	15
C. cumplimiento del Momento III	16

1 Introducción

Marvel Boxing es un videojuego de boxeo arcade desarrollado en C++ con interfaz gráfica construida sobre el framework Qt. La idea parte de un deporte conocido —el boxeo— y lo traslada a un universo de superhéroes donde los combates tienen una energía distinta: saltos amplios en la luna, arenas en vista cenital y rivales que aprenden de tus movimientos.

El equipo buscó que la arquitectura explicara por sí sola cómo se organizan las responsabilidades, cómo se gestiona la memoria y por qué se eligió cada contenedor o modelo físico.

Es un juego corto, directo y visualmente claro, El jugador puede elegir personaje, dificultad y nivel, luego lanzarse al combate contra un enemigo que toma sus propias decisiones. No es un juego complejo, pero sí demuestra que los conceptos del curso pueden combinarse en algo jugable.

2 Planteamiento del problema

El reto central del proyecto no era dibujar personajes en pantalla, sino construir un sistema que tuviera sentido desde adentro. Muchos prototipos de videojuegos funcionan pero son difíciles de explicar.

Desde el inicio se planteó el problema de cómo desarrollar un prototipo de combate con dos vistas diferentes, donde el jugador enfrenta a un enemigo autónomo, sin que el código se convierta en un bloque imposible de sostener.

La idea fue separar desde el comienzo la lógica del juego de su representación visual, y definir clases con responsabilidades claras antes de escribir una sola línea de Qt. Ese orden hizo posible integrar entrada por teclado, temporizador, física, colisiones, inteligencia artificial, sprites y menú sin que todo quedara enredado en el mismo lugar.

3 Definición general y objetivos

3.1 Idea general del videojuego

Marvel Boxing tiene dos niveles con dinámicas distintas:

- **Nivel 1 — Combate lunar (vista lateral):** el jugador y el enemigo pelean en un escenario lunar. La gravedad reducida hace que los saltos sean más amplios y las caídas más lentas. Las plataformas oscilan con movimiento senoidal, obligando al jugador a adaptarse constantemente.
- **Nivel 2 — Arena cenital (vista superior):** la pelea cambia de perspectiva. El movimiento ocurre en dos dimensiones dentro de un ring, y empiezan a caer objetos con efectos positivos o negativos. Aquí el enemigo usa el agente inteligente con mayor intensidad.

3.2 Objetivo general

Desarrollar un videojuego funcional en C++ y Qt que integre programación orientada a objetos, memoria dinámica, modelos físicos, dificultad configurable y un agente inteligente; cumpliendo los requisitos del Momento III del proyecto final.

3.3 Objetivos específicos

1. Implementar una arquitectura orientada a objetos con separación entre capa lógica y capa GUI.
2. Incluir al menos una jerarquía de herencia propia, independiente de las clases de Qt.
3. Administrar objetos dinámicos del juego mediante punteros y contenedores STL.
4. Incorporar al menos tres modelos físicos parametrizables distintos del movimiento rectilíneo simple.
5. Crear dos niveles con vistas y dinámicas claramente diferenciadas.
6. Desarrollar un agente inteligente con percepción, razonamiento, acción y aprendizaje.
7. Implementar dificultad configurable que modifique los parámetros internos del combate y de la IA.
8. Validar el funcionamiento mediante pruebas de controles, colisiones, cambio de nivel, dificultad y finalización de partida.

4 Especificación de requerimientos

4.1 Requisitos funcionales

Descripción	
1	El sistema debe permitir iniciar una partida desde un menú gráfico.
2	El sistema debe permitir seleccionar personaje del jugador y personaje rival.
3	El sistema debe permitir seleccionar nivel inicial.
4	El sistema debe permitir seleccionar dificultad: Entrenamiento, Normal o Vibranium.
5	El jugador debe poder moverse, saltar o desplazarse según el nivel activo.
6	El jugador debe poder atacar con golpe frontal y gancho.
7	El jugador debe poder bloquear para reducir el daño recibido.
8	El sistema debe actualizar vida, daño, retroceso, estados y resultado final.
9	El juego debe incluir dos niveles con vistas diferenciadas: lateral y cenital.

Descripción	
10	El enemigo debe tomar decisiones autónomas mediante un agente inteligente.
11	El sistema debe generar objetos con efectos dentro del segundo nivel.
12	El juego debe permitir reiniciar el combate y cambiar de nivel durante la ejecución.

4.2 Requisitos no funcionales

Descripción	
1	El código debe estar organizado en clases con responsabilidades claras.
2	La interfaz debe ser usable e identificar vida, personajes, nivel, dificultad y resultado.
3	La implementación debe usar Qt Widgets como framework de interfaz gráfica.
4	La lógica debe actualizarse con temporizador para mantener la simulación en tiempo real.
5	Los recursos gráficos deben cargarse desde el sistema de recursos de Qt cuando aplique.
6	El código debe incluir validaciones y excepciones para parámetros inválidos.
7	El repositorio debe evidenciar el avance mediante ramas y commits regulares.

4.3 Restricciones y dependencias

El proyecto se desarrolló en C++17 usando Qt Widgets. El archivo `MarvelBoxing.pro` incluye los módulos de Qt necesarios, los archivos fuente, cabeceras y el archivo de recursos `resources.qrc`. La compilación se realizó en un entorno compatible con Qt 6.10.2 y MinGW de 64 bits. También se incluyó una alternativa de compilación con CMake.

Las dependencias principales son: Qt Widgets (ventanas, eventos, botones, etiquetas, combo boxes y dibujo), Qt Core (temporizadores, cadenas, mapas y utilidades), Qt Gui (QPainter, QPixmap, QImage y eventos de teclado) y contenedores STL como `vector`, `deque`, `map` y `unordered_map`.

5 Metodología y planificación

El equipo trabajó de forma incremental. Primero se definió el alcance y la idea general del juego; después se construyó la arquitectura de clases de la capa lógica; finalmente se

integró la interfaz gráfica, los sprites, los niveles, la dificultad, el agente inteligente y las pruebas.

Fase	Actividades realizadas
Momento I	Definición del deporte base, universo especial, dos niveles, dinámicas, físicas, características del protagonista y agente autónomo inicial.
Momento II	Diseño de clases de la capa lógica, descripción de niveles, sprites, interacciones y arquitectura del agente inteligente.
Momento III	Implementación en Qt, integración de GUI, animaciones, niveles, dificultad, memoria dinámica, excepciones, pruebas, ejecutable, tráiler, video e informe final.

El control de versiones se manejó mediante Git con ramas de trabajo. Para la entrega final se debe verificar que el historial muestre commits frecuentes y que el código fuente, recursos y ejecutable queden disponibles en el repositorio.

6 Diseño y arquitectura del sistema

6.1 Separación por capas

Una de las primeras decisiones de diseño fue separar con claridad lo que el juego *hace* de lo que el juego *muestra*. Esta distinción que parece obvia pero cuesta mantener cuando el proyecto crece se materializó en dos capas:

- **Capa lógica** (`src/model`): contiene las reglas del juego, personajes, física, combate, objetos, efectos, niveles, control y agente inteligente. Esta capa no sabe nada de cómo se dibuja en pantalla.
- **Capa GUI** (`src/MainWindow.*` y `src/GameWidget.*`): contiene la ventana principal, el menú, la pantalla de juego, el dibujo, las animaciones, los eventos de teclado y el temporizador de actualización.

La clase `GestorNivel` coordina la simulación y `GameWidget` se encarga de visualizarla y actualizarla en pantalla.

6.2 Descripción de clases principales

Clase	Responsabilidad principal
MainWindow	Construye las pantallas de introducción, menú, selección de personajes, nivel, dificultad y pantalla de juego.
GameWidget	Recibe eventos de teclado, ejecuta el <code>QTimer</code> , llama al gestor de nivel, anima sprites y dibuja el HUD.
GestorNivel	Administra el estado del juego: nivel activo, personajes, objetos, plataformas, tiempo, dificultad, colisiones y resultado.
NivelBase	Define la interfaz común de los niveles mediante métodos virtuales puros.
NivelUnoLunar	Implementa la lógica del nivel lateral con gravedad lunar y plataformas oscilantes.
NivelDosCenital	Implementa la lógica del nivel cenital con ring, objetos que caen y decisiones de IA.
Personaje	Clase base de entidades combatientes: vida, movimiento, ataque, bloqueo, daño y estado.
Jugador	Especializa <code>Personaje</code> para procesar la entrada del usuario y administrar la super habilidad.
Enemigo	Especializa <code>Personaje</code> e integra el agente inteligente para decidir acciones.
Fisica	Centraliza gravedad, caída de objetos, oscilación de plataformas y detección por distancia.
SistemaCombate	Evalúa impactos, calcula daño y aplica reducción por bloqueo.
Objeto	Representa elementos del nivel 2 (tomate, bebida) con caída y efecto asociado.
Efecto	Aplica modificadores temporales de velocidad y ataque sobre un personaje.
AgenteInteligente	Ejecuta el ciclo percepción → razonamiento → acción → aprendizaje.
MemoriaIA	Guarda una ventana de acciones recientes del jugador y calcula frecuencias para detectar patrones.

7 Desarrollo e implementación

7.1 Programación orientada a objetos

La jerarquía de herencia propia aparece en dos lugares. El primero es la relación `Personaje` → {`Jugador`, `Enemigo`}: evita duplicar atributos como vida, posición, velocidad, dirección, estado y métodos comunes de movimiento o ataque. El jugador agrega comportamiento de entrada y super habilidad; el enemigo agrega la toma de decisiones autónoma.

El segundo es la relación `NivelBase` $\rightarrow$ `{NivelUnoLunar, NivelDosCenital}`, que permite manejar ambos niveles con una interfaz común: `cargar`, `actualizar`, `numero` y `nombre`. Esta separación cumple el principio de responsabilidad única: la base contiene lo común y las clases derivadas contienen las diferencias concretas.

7.2 Memoria dinámica

El juego usa memoria dinámica principalmente en `GestorNivel`. Los personajes, objetos y plataformas se almacenan en vectores de punteros:

```
1 std::vector<Personaje*> m_personajes;
2 std::vector<Objeto*>    m_objetos;
3 std::vector<Plataforma*> m_plataformas;
```

Listing 1: Contenedores de entidades en `GestorNivel`.

El uso de punteros permite almacenamiento polimórfico: el vector de personajes puede contener tanto `Jugador` como `Enemigo`. Para evitar fugas de memoria, `GestorNivel` implementa el método `limpiar()`:

```
1 void GestorNivel::limpiar()
2 {
3     for (Personaje* p : m_personajes) delete p;
4     for (Objeto*    o : m_objetos)    delete o;
5     for (Plataforma* pl : m_plataformas) delete pl;
6     m_personajes.clear();
7     m_objetos.clear();
8     m_plataformas.clear();
9 }
```

Listing 2: Liberación de memoria dinámica en `GestorNivel`.

Los objetos inactivos se retiran durante cada ciclo mediante `std::remove_if`, lo que evita que sigan participando en colisiones o acumulando memoria innecesariamente.

7.3 Contenedores seleccionados y justificación

Contenedor	Uso	Justificación
Personaje	Lista de jugador y enemigo	Acceso por índice, recorrido simple y polimorfismo.
Objeto	Objetos activos del nivel 2	Se recorren todos cada ciclo y se eliminan inactivos.
Plataforma	Plataformas del nivel lunar	Cantidad pequeña, recorrido secuencial.
TipoAccion	Historial reciente de la IA	Inserción al final y retiro al inicio eficientes para mantener la ventana de aprendizaje.
TipoAccion,int	Frecuencia de acciones	Facilita contar y detectar patrones repetidos.

Contenedor	Uso	Justificación
QMap	Botones y animaciones GUI	Integración natural con tipos Qt como <code>QString</code> .
QHash	Caché de bounds de sprites	Acceso rápido a cálculos repetidos en el dibujo.

7.4 Interfaz gráfica y conexiones en Qt

`MainWindow` construye todas las pantallas de menú y selección. `GameWidget` recibe eventos de teclado mediante `keyPressEvent` y `keyReleaseEvent`, y actualiza la simulación con un `QTimer`. El método `tick()` calcula el tiempo transcurrido, actualiza la lógica y solicita repintado mediante `update()`.

Las conexiones se establecen con la sintaxis de señales y slots de Qt, lo que evita consultar constantemente el estado de los botones:

```

1 connect(boton, &QPushButton::clicked, this, [this]() {
2     iniciarDesdeMenu();
3 });

```

Listing 3: Estructura general de una conexión en Qt.

8 Modelos físicos implementados

El proyecto incluye tres modelos físicos principales, todos parametrizables y con efecto visible en pantalla. El movimiento rectilíneo simple no se cuenta como modelo físico principal.

8.1 Gravedad lunar con flotación

En el nivel 1 se aplica una gravedad menor que la terrestre para que los saltos se sientan más amplios y lentos, coherentes con el escenario lunar:

Donde g es la gravedad configurada (2.35), k_f es el factor de flotación (0.88), C es un factor de escala arcade y Δt es el paso de tiempo.

8.2 Movimiento oscilatorio de plataformas

Las plataformas del nivel lunar se mueven con oscilación senoidal, introduciendo una perturbación que el jugador debe anticipar:

Donde y_0 es la posición base, A es la amplitud y T es el periodo. Cada plataforma tiene parámetros propios

8.3 Caída vertical de objetos con altura simulada

En el nivel 2, los objetos caen desde una altura simulada. Mientras descenden, el sprite puede dibujarse separado de su sombra para anticipar el punto de impacto:

Donde h es la altura simulada y v_c es la velocidad de caída (aprox. 145.0 unidades/s).

8.4 Colisiones y rango de impacto

La detección de golpes y recolección de objetos usa distancia euclidiana:

En el nivel 2 se ajusta el centro de ataque para simular el alcance del puño en vista cenital.

9 Agente inteligente

El enemigo usa una arquitera de percepción, razonamiento, ejecución de acción y aprendizaje. Esta división permite explicar claramente cada componente sin que la lógica de la IA quede enterrada en un bloque de condicionales.

9.1 Percepción

La estructura `Percepcion` captura el estado relevante del entorno: posición y vida del jugador y del enemigo, distancia entre ellos, acción reciente del jugador, nivel activo, estados de ambos personajes, objetos cercanos y si los personajes están en el suelo.

9.2 Razonamiento

La clase `Razonador` evalúa la percepción, la memoria y la dificultad. Su salida es una `DecisionIA`: moverse, atacar, bloquear, esquivar o recoger un objeto. Algunos ejemplos del razonamiento:

- Si el enemigo tiene poca vida y hay una bebida energética cerca, la prioriza.
- Si el jugador está bloqueando y está cerca, intenta un gancho para castigar la guardia.
- Si el jugador está vulnerable o saltando, ataca.
- Si detecta un patrón de ataque repetido, cambia su respuesta.

9.3 Acción

La acción se ejecuta directamente sobre el enemigo: si la decisión es moverse, calcula una dirección hacia el destino; si es atacar, llama los métodos heredados de `Personaje`; si es bloquear o esquivar, modifica su estado para responder al siguiente golpe.

9.4 Aprendizaje

MemoriaIA usa un **deque** para mantener una ventana de acciones recientes del jugador y un **map** para contarlas. Cuando una acción supera cierta frecuencia, el agente la identifica como patrón y ajusta sus decisiones:

```

1 void MemoriaIA::registrarAccion(TipoAccion accion)
2 {
3     m_historialAcciones.push_back(accion);
4     m_frecuenciaAtaques[accion]++;
5
6     if ((int)m_historialAcciones.size() >
7         m_ventanaAprendizaje) {
8         TipoAccion descartada = m_historialAcciones.front();
9         m_historialAcciones.pop_front();
10        m_frecuenciaAtaques[descartada] =
11            std::max(0, m_frecuenciaAtaques[descartada] - 1);
12    }
13 }
```

Listing 4: Núcleo del aprendizaje de la IA.

10 Dificultad

La dificultad no se reduce a quitar tiempo o aumentar obstáculos: modifica directamente los parámetros internos del agente inteligente.

Dificultad	Efecto esperado
Entrenamiento	El agente reacciona con menor presión, decide con más pausa y da mayor margen de recuperación al jugador.
Normal	Balance intermedio entre agresividad, defensa, persecución y uso de objetos.
Vibranium	El agente aumenta agresividad, rango efectivo, interés por objetos, frecuencia de combos y capacidad defensiva.

La abstracción de dificultad se concentra en perfiles internos de la IA donde se parametrizan variables como agresividad, rango de ataque, distancia de defensa, interés por objetos, pausa de decisión y frecuencia de golpes.

11 Cambios frente a los momentos anteriores

El Momento I planteó la idea general y las dinámicas. El Momento II presentó la arquitectura lógica y la estructura del agente. En el Momento III se realizaron los siguientes cambios:

- Se agregó la capa GUI completa con `MainWindow` y `GameWidget`.
- Se implementó el menú con selección de personajes, nivel y dificultad.
- Se separó la lógica de cada nivel en `NivelUnoLunar` y `NivelDosCenital`.
- Se amplió el agente inteligente con estructuras explícitas de percepción, razonamiento, memoria y decisión.
- Se agregaron sprites animados y seleccionables para los personajes.
- Se implementaron objetos con efectos temporales sobre velocidad y ataque.
- Se agregó una super habilidad con barra de carga.
- Se incorporaron validaciones con excepciones en las clases principales.
- Se ajustó la dificultad para afectar parámetros internos del agente, no solo el tiempo de partida.

Estos cambios no reemplazaron la idea inicial sino que la volvieron concreta y jugable.

12 Procedimientos de prueba

12.1 Plan de pruebas

N.º	Prueba	Resultado esperado
P-01	Iniciar el juego desde el menú.	Se abre la pantalla de juego con personaje, rival, nivel y dificultad seleccionados.
P-02	Mover al jugador con las teclas definidas.	El jugador cambia de posición y la animación corresponde al movimiento.
P-03	Saltar en el nivel lunar.	El personaje realiza un salto amplio y cae lentamente.
P-04	Verificar plataformas oscilantes.	Las plataformas se mueven suavemente y permiten apoyo al caer sobre ellas.
P-05	Atacar con golpe frontal y gancho.	El enemigo recibe daño si está en rango; si no, el golpe falla.
P-06	Bloquear con tecla correspondiente.	El daño recibido se reduce durante la ventana de bloqueo.
P-07	Cambiar al nivel cenital.	El movimiento cambia a vista superior y los límites del ring se aplican.

N.º	Prueba	Resultado esperado
P-08	Esperar generación de objetos.	Aparecen tomates o bebidas, caen con sombra y aplican efectos al recogerlos.
P-09	Comparar dificultades.	La IA cambia agresividad, respuesta y presión de combate.
P-10	Finalizar partida.	El sistema muestra victoria, derrota o empate según vida y tiempo.
P-11	Reiniciar nivel.	Se limpian objetos anteriores, se reubican personajes y no quedan elementos activos.
P-12	Probar valores inválidos.	Las clases lanzan excepciones ante gravedad, vida, daño o nivel inválido.

12.2 Resultados de prueba

Las pruebas manuales verificaron que el flujo principal funciona: menú, selección, combate lateral, combate cenital, ataques, bloqueo, objetos, agente inteligente, cambio de nivel y reinicio. Se confirmó además que los objetos dinámicos se eliminan correctamente al limpiar el nivel o al quedar inactivos.

12.3 Controles

Tecla	Acción
A / D	Mover izquierda y derecha
W	Saltar (nivel lunar) o subir (vista cenital)
S	Bloquear (nivel lunar) o bajar (vista cenital)
J	Golpe frontal
K	Gancho
L	Cubrirse para reducir daño
R	Reiniciar combate
N	Cambiar de nivel

13 Resultados y discusión

La versión final logra una experiencia jugable con dos niveles diferenciados. El primer nivel se enfoca en combate lateral, saltos y plataformas; el segundo cambia la perspectiva a vista cenital, incorpora objetos y pone a prueba al jugador frente a un agente más activo.

Desde el punto de vista técnico, el proyecto evidencia POO mediante clases especializadas y herencia propia; memoria dinámica mediante vectores de punteros; contenedores STL para entidades y memoria de IA; excepciones para validar parámetros; modelos físicos parametrizables; y separación de responsabilidades entre lógica y GUI.

Las dificultades más importantes durante el desarrollo fueron la integración de sprites con estados lógicos, el ajuste de rangos de golpe, la sincronización entre el temporizador y las animaciones, y el balance de la IA para que sea desafiante sin volverse injusta. También fue necesario cuidar que el reinicio o cambio de nivel limpiara correctamente los objetos creados dinámicamente.

Como limitación actual, el sonido puede requerir ajuste o integración final según los audios libres de derechos elegidos por el equipo. Se recomienda también verificar el ejecutable en un equipo diferente para confirmar que todas las DLL y recursos de Qt se despliegan correctamente.

14 Conclusiones

El desarrollo de Marvel Boxing permitió integrar los temas principales del curso en un proyecto funcional. La división en clases facilitó organizar el código y explicar con claridad las responsabilidades de cada componente.

La jerarquía Personaje Jugador, Enemigo ayudó a reutilizar atributos y comportamientos comunes, mientras que NivelBase permitió separar la lógica de cada escenario sin duplicar código. Eso fue una ventaja real cuando llegó el momento de agregar el nivel cenital: no hubo que tocar nada de lo que ya funcionaba en el lunar.

El uso de memoria dinámica fue necesario para manejar entidades creadas durante la partida, pero también obligó a pensar en el ciclo de vida de cada objeto. Implementar `limpiar()`, fue la parte del código que demuestra que los punteros se usaron conscientemente.

Los modelos físicos aportaron variedad real a la jugabilidad: la gravedad lunar cambia la sensación del salto, las plataformas oscilantes obligan al jugador a leer el escenario, y la caída de objetos con sombra anticipada añade un elemento de decisión en el nivel 2. Todos son parametrizables y producen efectos visibles, lo que los hace más defendibles que un simple desplazamiento en línea recta.

El agente inteligente es simple, pero funciona: percibe, razona, actúa y recuerda. Eso es suficiente para que la IA no se sienta como un enemigo que solo carga de frente, y es lo suficientemente claro para explicarse en cinco minutos durante la sustentación.

Como mejoras futuras se propone fortalecer el audio, ampliar animaciones, agregar efectos visuales al impacto, guardar puntajes y refinar el balance de dificultad con pruebas de usuarios reales.

14 Referencias

1. Qt Project. *Qt Widgets Documentation*. Documentación oficial de Qt. Consultado para el uso de ventanas, widgets, eventos, QPainter, QTimer y recursos.
2. cppreference. *C++ Standard Library Reference*. Consultado para `std::vector`, `std::deque`, `std::map`, excepciones y utilidades de C++17.
3. Material de clase de Informática II. Diapositivas, videos y pautas del proyecto final sobre POO, memoria dinámica, GUI, repositorios, diagrama de clases y agente

inteligente.

4. Repositorio del proyecto Marvel Boxing. Código fuente, recursos gráficos, historial de commits, ejecutable y documentación del equipo. [Completar: pegar enlace del repositorio]

A Estructura de carpetas

```
MarvelBoxing/
|-- MarvelBoxing.pro
|-- CMakeLists.txt
|-- resources.qrc
|-- assets_menu/
|-- src/
    |-- main.cpp
    |-- MainWindow.h / MainWindow.cpp
    |-- GameWidget.h / GameWidget.cpp
    |-- model/
        |-- Personaje.h / Personaje.cpp
        |-- GestorNivel.h / GestorNivel.cpp
        |-- Fisica.h / Fisica.cpp
        |-- SistemaCombate.h / SistemaCombate.cpp
        |-- AgenteInteligente.h / AgenteInteligente.cpp
        |-- Objeto.h / Objeto.cpp
        |-- Efecto.h / Efecto.cpp
        |-- Plataforma.h / Plataforma.cpp
        |-- Control.h / Control.cpp
        |-- niveles/
            |-- NivelBase.h
            |-- NivelUnoLunar.h / NivelUnoLunar.cpp
            |-- NivelDosCenital.h / NivelDosCenital.cpp
```

B Fragmento representativo de herencia propia

```
1 class Personaje {
2 public:
3     Personaje(QString nombre, int vida,
4               float velocidadBase, Vector2D posicion);
5     virtual ~Personaje() = default;
6     virtual void actualizar(float dt);
7     virtual void mover(const Vector2D& direccion, float dt);
8     virtual void atacar(TipoAccion tipo);
9 };
10
11 class Jugador : public Personaje {
12 public:
13     Jugador(QString nombre, Vector2D posicion);
14     void procesarEntrada(const Control& input, bool topDown);
```



```

15     void actualizar(float dt) override;
16 };
17
18 class Enemigo : public Personaje {
19 public:
20     Enemigo(QString nombre, Vector2D posicion);
21     void tomarDecision(const Jugador& jugador,
22                       const GestorNivel& nivel,
23                       Dificultad dificultad);
24 };

```

Listing 5: Jerarquía Personaje, Jugador y Enemigo.

C cumplimiento del Momento III

Requisito	Estado	Evidencia
Tres modelos físicos (sin movimiento rectilíneo).	Cumple	Gravedad lunar, oscilación senoidal, caída de objetos, colisiones.
POO y memoria dinámica obligatoria.	Cumple	Clases del modelo, vectores de punteros, método <code>limpiar</code> .
Diagrama de clases lógica + GUI.	Cumple	Figuras del informe.
Versión resumen del diagrama.	Cumple	Figura resumen para sustentación.
Herencia propia no Qt.	Cumple	Jugador/Enemigo de Personaje; niveles de NivelBase.
Dificultad no trivial.	Cumple	Perfiles de IA con parámetros de agresividad, rango y defensa.
Excepciones.	Cumple	Validaciones en Personaje, Fisica, GestorNivel.
GUI en Qt.	Cumple	MainWindow, GameWidget, widgets y QTimer.
Agente inteligente.	Cumple	AgenteInteligente, Percepcion, Razonador, MemoriaIA, DecisionIA.